PROGRAMAÇÃO ORIENTADA A OBJETOS

Existem diferentes paradigmas de programação. Conforme você aprende outras linguagens, começará a reconhecer padrões como estes. Até este ponto, você trabalhou de forma procedural, passo a passo. A programação orientada a objetos (OOP) é uma solução convincente para problemas relacionados à programação. Para começar, digite o código student.py na janela do terminal e o código a seguir:

```
name = input("Name: ")
house = input("House: ")
print(f"{name} from {house}")
```

Observe que este programa segue um paradigma procedural, passo a passo: Assim como você viu em partes anteriores deste curso.

Baseando-nos em nosso trabalho das semanas anteriores, podemos criar funções para abstrair partes deste programa.

```
def main():
    name = get_name()
    house = get_house()
    print(f"{name} from {house}")


def get_name():
    return input("Name: ")


def get_house():
    return input("House: ")


if __name__ == "__main__":
```

```
main()
```

Observe como get_name e get_house abstraem algumas das necessidades de nossa função principal. Além disso, observe como as últimas linhas do código acima dizem ao compilador para executar a função principal.

Podemos simplificar ainda mais nosso programa armazenando o aluno como uma tupla. Uma tupla é uma sequência de valores. Ao contrário de uma lista, uma tupla não pode ser modificada. Em espírito, estamos retornando dois valores.

```
def main():
    name, house = get_student()
    print(f"{name} from {house}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return name, house


if __name__ == "__main__":
    main()
```

Observe como get_student retorna name, house.

Empacotando essa tupla, de modo que possamos retornar ambos os itens para uma variável chamada student, podemos modificar nosso código da seguinte forma.

```
def main():
    student = get_student()
    print(f"{student[0]} from {student[1]}")
```

```
def get_student():
    name = input("Name: ")
    house = input("House: ")
    return (name, house)


if __name__ == "__main__":
    main()
```

Observe que (name, house) explicitamente informa a qualquer pessoa lendo nosso código que estamos retornando dois valores em um. Além disso, observe como podemos indexar em tuplas usando student[0] ou student[1].

As tuplas são imutáveis, o que significa que não podemos alterar esses valores. A imutabilidade é uma maneira pela qual podemos programar defensivamente.

```
def main():
    student = get_student()
    if student[0] == "Padma":
        student[1] = "Ravenclaw"
    print(f"{student[0]} from {student[1]}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return name, house


if __name__ == "__main__":
    main()
```

Observe que este código produz um erro. Como as tuplas são imutáveis, não podemos reatribuir o valor de student[1].

Se quisermos fornecer flexibilidade aos nossos colegas programadores, podemos utilizar uma lista da seguinte forma.

```
def main():
    student = get_student()
    if student[0] == "Padma":
        student[1] = "Ravenclaw"
    print(f"{student[0]} from {student[1]}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return [name, house]


if __name__ == "__main__":
    main()
```

Observe que as listas são mutáveis. Ou seja, a ordem de house e name pode ser trocada por um programador. Você pode decidir usar isso em alguns casos em que deseja fornecer mais flexibilidade com o custo da segurança do seu código. Afinal, se a ordem desses valores for modificável, os programadores que trabalham com você podem cometer erros mais adiante.

Um dicionário também poderia ser utilizado nesta implementação. Lembre-se de que os dicionários fornecem um par chave-valor.

```
def main():
    student = get_student()
    print(f"{student['name']} from {student['house']}")
```

```
def get_student():
    student = {}
    student["name"] = input("Name: ")
    student["house"] = input("House: ")
    return student


if __name__ == "__main__":
    main()
```

Observe que, neste caso, dois pares chave-valor são retornados. Uma vantagem dessa abordagem é que podemos indexar este dicionário usando as chaves.

Ainda assim, nosso código pode ser melhorado ainda mais. Observe que há uma variável desnecessária. Podemos remover student = {} porque não precisamos criar um dicionário vazio.

```
def main():
    student = get_student()
    print(f"{student['name']} from {student['house']}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return {"name": name, "house": house}


if __name__ == "__main__":
    main()
```

Observe que podemos usar chaves {} na instrução de retorno para criar o dicionário e retorná-lo tudo na mesma linha.

Podemos fornecer nosso caso especial com Padma em nossa versão de dicionário de código como segue.

```
def main():
    student = get_student()
    if student["name"] == "Padma":
        student["house"] = "Ravenclaw"
    print(f"{student['name']} from {student['house']}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return {"name": name, "house": house}


if __name__ == "__main__":
    main()
```

Observe como, de forma semelhante ao espírito de nossas iterações anteriores deste código, podemos utilizar os nomes das chaves para indexar nosso dicionário de student.

CLASSES

Classes são uma maneira pela qual, na programação orientada a objetos, podemos criar nosso próprio tipo de dado e dar a eles nomes. Uma classe é como um molde para um tipo de dado - onde podemos inventar nosso próprio tipo de dado e dar a eles um nome. Podemos modificar nosso código da seguinte maneira para implementar nossa própria classe chamada Student:

```
class Student:
    ...


def main():
    student = get_student()
    print(f"{student.name} from {student.house}")


def get_student():
    student = Student()
    student.name = input("Name: ")
    student.house = input("House: ")
    return student


if __name__ == "__main__":
    main()
```

Observe por convenção que Student está em maiúscula. Além disso, observe que o ... simplesmente significa que mais tarde voltaremos para concluir essa parte do nosso código. Além disso, observe que em get_student, podemos criar um estudante da classe Student usando a sintaxe student = Student(). Além disso, observe que utilizamos a "notação de ponto" para acessar atributos dessa variável student da classe Student.

Toda vez que você cria uma classe e utiliza esse modelo para criar algo, você cria o que é chamado de "objeto" ou "instância". No caso do nosso código, student é um objeto. Além disso, podemos estabelecer algumas bases para os atributos esperados dentro de um objeto cuja classe é Student. Podemos modificar nosso código da seguinte maneira:

```
class Student:
    def __init__(self, name, house):
        self.name = name
```

```
    self.house = house


def main():
    student = get_student()
    print(f"{student.name} from {student.house}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    student = Student(name, house)
    return student


if __name__ == "__main__":
    main()
```

Observe que dentro de Student, padronizamos os atributos desta classe. Podemos criar uma função dentro da classe Student, chamada de "método", que determina o comportamento de um objeto da classe Student. Dentro desta função, ela recebe o nome e a casa passados para ela e atribui essas variáveis a este objeto. Além disso, observe como o construtor student = Student(name, house) chama esta função dentro da classe Student e cria um estudante. self refere-se ao objeto atual que acabou de ser criado.

Podemos simplificar nosso código da seguinte maneira:

```
class Student:
    def __init__(self, name, house):
        self.name = name
        self.house = house
```

```
def main():
    student = get_student()
    print(f"{student.name} from {student.house}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return Student(name, house)


if __name__ == "__main__":
    main()
```

Observe como return Student(name, house) simplifica a iteração anterior de nosso código onde a declaração do construtor era executada em sua própria linha.

Raise

Programa orientado a objetos encoraja você a encapsular toda a funcionalidade de uma classe dentro da definição da classe. E se algo der errado? E se alguém tentar digitar algo aleatório? E se alguém tentar criar um aluno sem nome? Modifique seu código da seguinte maneira:

```
class Student:
    def __init__(self, name, house):
        if not name:
            raise ValueError("Missing name")
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]:
            raise ValueError("Invalid house")
        self.name = name
        self.house = house
```

```
def main():
    student = get_student()
    print(f"{student.name} from {student.house}")


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return Student(name, house)


if __name__ == "__main__":
    main()
```

Observe como agora verificamos se um nome é fornecido e se uma casa adequada é designada. Acontece que podemos criar nossas próprias exceções que alertam o programador para um erro potencial criado pelo usuário chamado raise. No caso acima, lançamos ValueError com uma mensagem de erro específica.

Acontece que o Python permite que você crie uma função específica pela qual você pode imprimir os atributos de um objeto. Modifique seu código da seguinte maneira:

```
class Student:
    def __init__(self, name, house, patronus):
        if not name:
            raise ValueError("Missing name")
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]:
            raise ValueError("Invalid house")
        self.name = name
        self.house = house
        self.patronus = patronus
```

```
    def __str__(self):
        return f"{self.name} from {self.house}"


def main():
    student = get_student()
    print(student)


def get_student():
    name = input("Name: ")
    house = input("House: ")
    patronus = input("Patronus: ")
    return Student(name, house, patronus)


if __name__ == "__main__":
    main()
```

Observe como def **str**(self) fornece um meio pelo qual um aluno é retornado quando chamado. Portanto, agora você, como programador, pode imprimir um objeto, seus atributos ou quase tudo o que desejar relacionado a esse objeto.

Str é um método integrado que vem com as classes Python. Acontece que podemos criar nossos próprios métodos para uma classe também! Modifique seu código da seguinte maneira:

```
class Student:
    def __init__(self, name, house, patronus=None):
        if not name:
            raise ValueError("Missing name")
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]:
```

```
        raise ValueError("Invalid house")
    if patronus and patronus not in ["Stag", "Otter", "Jack Russell terrier"]:
        raise ValueError("Invalid patronus")
    self.name = name
    self.house = house
    self.patronus = patronus

def __str__(self):
    return f"{self.name} from {self.house}"

def charm(self):
    match self.patronus:
        case "Stag":
            return "🐴"
        case "Otter":
            return "🦦"
        case "Jack Russell terrier":
            return "🐶"
        case _:
            return "🪄"
```

```
def main():
    student = get_student()
    print("Expecto Patronum!")
    print(student.charm())


def get_student():
    name = input("Name: ")
    house = input("House: ")
    patronus = input("Patronus: ") or None
    return Student(name, house, patronus)


if __name__ == "__main__":
    main()
```

Classes Classes são uma maneira pela qual, na programação orientada a objetos, podemos criar nosso próprio tipo de dados e dar-lhes nomes. Uma classe é como um molde para um tipo de dados - onde podemos inventar nosso próprio tipo de dados e dar-lhes um nome. Podemos modificar nosso código da seguinte forma para implementar nossa própria classe chamada Student:

```
class Student:
    def __init__(self, name, house):
        if not name:
            raise ValueError("Invalid name")
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]:
            raise ValueError("Invalid house")
        self.name = name
        self.house = house

    def __str__(self):
```

```
    return f"{self.name} from {self.house}"


def main():
    student = get_student()
    student.house = "Number Four, Privet Drive"
    print(student)


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return Student(name, house)


if __name__ == "__main__":
    main()
```

Observe como temos apenas dois métodos: _init_ e _str_.

Decoradores

Propriedades podem ser utilizadas para fortalecer nosso código. Em Python, definimos propriedades usando "decoradores" de função, que começam com @. Modifique seu código da seguinte forma:

```
class Student:
    def __init__(self, name, house):
        if not name:
            raise ValueError("Invalid name")
        self.name = name
```

```
        self.house = house

    def __str__(self):
        return f"{self.name} from {self.house}"

    # Getter for house
    @property
    def house(self):
        return self._house

    # Setter for house
    @house.setter
    def house(self, house):
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]:
            raise ValueError("Invalid house")
        self._house = house


def main():
    student = get_student()
    print(student)


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return Student(name, house)


if __name__ == "__main__":
```

```
main()
```

Observe como escrevemos @property acima de uma função chamada casa. Ao fazer isso, definimos casa como uma propriedade de nossa classe. Com casa como uma propriedade, ganhamos a capacidade de definir como algum atributo de nossa classe, _casa, deve ser definido e recuperado. De fato, agora podemos definir uma função chamada "setter", via @house.setter, que será chamada sempre que a propriedade casa for definida, por exemplo, com student.house = "Gryffindor". Aqui, fizemos nosso setter validar valores de casa para nós. Observe como levantamos um ValueError se o valor de casa não for uma das casas de Harry Potter, caso contrário, usaremos casa para atualizar o valor de _casa. Por que _casa e não casa? casa é uma propriedade de nossa classe, com funções através das quais um usuário tenta definir nosso atributo de classe. _casa é o próprio atributo de classe. O sublinhado inicial, _, indica aos usuários que eles não precisam (e na verdade, não devem!) modificar este valor diretamente. _casa só deve ser definido através do setter de casa. Observe como a propriedade casa simplesmente retorna esse valor de _casa, nosso atributo de classe que presumivelmente foi validado usando nosso setter de casa. Quando um usuário chama estudante.casa, eles estão obtendo o valor de _casa através de nosso getter de casa.

Além do nome da casa, também podemos proteger o nome de nosso estudante. Modifique seu código da seguinte forma:

```
class Student:
    def __init__(self, name, house):
        self.name = name
        self.house = house

    def __str__(self):
        return f"{self.name} from {self.house}"

    # Getter for name
    @property
    def name(self):
        return self._name

    # Setter for name
    @name.setter
    def name(self, name):
        if not name:
            raise ValueError("Invalid name")
```

```
        self._name = name

    @property
    def house(self):
        return self._house

    @house.setter
    def house(self, house):
        if house not in ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]:
            raise ValueError("Invalid house")
        self._house = house


def main():
    student = get_student()
    print(student)


def get_student():
    name = input("Name: ")
    house = input("House: ")
    return Student(name, house)


if __name__ == "__main__":
    main()
```

Observe que, assim como no código anterior, fornecemos um getter e um setter para o nome.

Conectando-se ao Trabalho Anterior neste Curso

Tradução livre do material ofertado no curso CS50's Introduction to Programming with Python

Embora não seja explicitamente declarado nas partes anteriores deste curso, você tem usado classes e objetos o tempo todo. Se você investigar a documentação de int, verá que é uma classe com um construtor. É um modelo para criar objetos do tipo int. Você pode aprender mais na documentação de int do Python. As strings também são uma classe. Se você usou str.lower(), estava usando um método que veio dentro da classe str. Ao olhar para a documentação de list, você pode ver os métodos contidos nela, como list.append(). Para ver como você vem usando classes o tempo todo, vá para o seu console e digite o código type.py e depois o código a seguir:

```
print(type(50))
```

Observe como executando este código, ele exibirá que a classe de 50 é int.

Também podemos aplicar isso a str da seguinte forma:

```
print(type("hello, world"))
```

Observe como a execução deste código indicará que isso é da classe str.

Também podemos aplicar isso a list da seguinte forma:

```
print(type([]))
```

Observe como a execução deste código indicará que isso é da classe list.

Também podemos aplicar isso a uma lista usando o nome da classe list integrada do Python da seguinte forma:

```
print(type(list()))
```

Observe como a execução deste código indicará que isso é da classe list.

Também podemos aplicar isso a dict da seguinte forma:

```
print(type({}))
```

Observe como a execução deste código indicará que isso é da classe dict.

Também podemos aplicar isso a um dict usando o nome da classe dict integrada do Python da seguinte forma:

```
print(type(dict()))
```

Observe como a execução deste código indicará que isso é da classe dict.

MÉTODOS DE CLASSE

Às vezes, queremos adicionar funcionalidade à própria classe, não às instâncias dessa classe. @classmethod é uma função que podemos usar para adicionar funcionalidade a uma classe como um todo. Aqui está um exemplo de não usar um método de classe. Na janela do terminal, digite o code hat.py e o código a seguir:

```
import random


class Hat:
    def __init__(self):
        self.houses = ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]

    def sort(self, name):
        print(name, "is in", random.choice(self.houses))


hat = Hat()
hat.sort("Harry")
```

Observe como quando passamos o nome do aluno para o chapéu seletor, ele nos dirá qual casa está atribuída ao aluno. Observe que hat = Hat() instancia um chapéu. A funcionalidade de classificação é sempre controlada pela instância da classe Hat. Ao executar hat.sort("Harry"), passamos o nome do aluno para o método sort da instância específica de Hat, que chamamos de hat.

Podemos querer, no entanto, executar a função sort sem criar uma instância específica do chapéu seletor (afinal, só existe uma!). Podemos modificar nosso código da seguinte maneira:

```
import random


class Hat:

    houses = ["Gryffindor", "Hufflepuff", "Ravenclaw", "Slytherin"]

    @classmethod
    def sort(cls, name):
        print(name, "is in", random.choice(cls.houses))


Hat.sort("Harry")
```

Observe como o método __init__ foi removido porque não precisamos instanciar um chapéu em nenhum lugar do nosso código. self, portanto, não é mais relevante e é removido. Especificamos essa classificação como @classmethod, substituindo self por cls. Finalmente, observe como Hat é maiúsculo por convenção perto do final deste código, porque este é o nome da nossa classe.

Voltando ao Students.py podemos modificar nosso código da seguinte forma, abordando algumas oportunidades perdidas relacionadas a @classmethods:

```
class Student:
    def __init__(self, name, house):
        self.name = name
        self.house = house

    def __str__(self):
        return f"{self.name} from {self.house}"

    @classmethod
    def get(cls):
```

```
    name = input("Name: ")
    house = input("House: ")
    return cls(name, house)


def main():
  student = Student.get()
  print(student)


if __name__ == "__main__":
  main()
```

Observe que get_student é removido e um @classmethod chamado get é criado. Este método agora pode ser chamado sem a necessidade de criar um aluno primeiro.

MÉTODOS ESTÁTICOS

Acontece que além de @classmethods, que são distintos dos métodos de instância, existem outros tipos de métodos também.

Usar @staticmethod pode ser algo que você queira explorar. Embora não seja abordado explicitamente neste curso, você pode aprender mais sobre métodos estáticos e sua distinção dos métodos de classe.

HERANÇA

A herança é, talvez, o recurso mais poderoso da programação orientada a objetos.

Acontece que você pode criar uma classe que “herda” métodos, variáveis e atributos de outra classe.

No terminal, execute o código wizard.py. Codifique da seguinte forma:

```
class Wizard:
  def __init__(self, name):
```

```
        if not name:
            raise ValueError("Missing name")
        self.name = name

    ...

class Student(Wizard):
    def __init__(self, name, house):
        super().__init__(name)
        self.house = house

    ...

class Professor(Wizard):
    def __init__(self, name, subject):
        super().__init__(name)
        self.subject = subject

    ...

wizard = Wizard("Albus")
student = Student("Harry", "Gryffindor")
professor = Professor("Severus", "Defense Against the Dark Arts")
...
```

Observe que existe uma classe acima chamada Wizard e uma classe chamada Student. Além disso, observe que existe uma classe chamada Professor. Alunos e professores têm nomes. Além disso, tanto alunos quanto professores são magos. Portanto, tanto o Aluno quanto o Professor herdam as características do Wizard. Dentro da classe “filho”, Student, Student pode herdar da classe “pai” ou “super” Wizard, pois a linha super().__init__(name) executa o método init do Wizard. Por fim, observe que as últimas linhas deste código criam um bruxo chamado Alvo, um aluno chamado Harry e assim por diante.

HERANÇA E EXCEÇÕES

Embora tenhamos acabado de introduzir a herança, temos usado isso o tempo todo durante o uso de exceções.

Acontece que as exceções surgem em uma hierarquia, onde há classes de filhos, pais e avós. Eles são ilustrados abaixo:

```
BaseException
 +-- KeyboardInterrupt
 +-- Exception
      +-- ArithmeticError
      |    +-- ZeroDivisionError
      +-- AssertionError
      +-- AttributeError
      +-- EOFError
      +-- ImportError
      |    +-- ModuleNotFoundError
      +-- LookupError
      |    +-- KeyError
      +-- NameError
      +-- SyntaxError
      |    +-- IndentationError
      +-- ValueError
 ...
```

SOBRECARGA DO OPERADOR

Alguns operadores como + e - podem ser “sobrecarregados” de modo que possam ter mais habilidades além da simples aritmética.

Na janela do terminal, digite o código vault.py. Em seguida, codifique da seguinte forma:

```
class Vault:
    def __init__(self, galleons=0, sickles=0, knuts=0):
        self.galleons = galleons
        self.sickles = sickles
        self.knuts = knuts

    def __str__(self):
        return f"{self.galleons} Galleons, {self.sickles} Sickles, {self.knuts} Knuts"

    def __add__(self, other):
        galleons = self.galleons + other.galleons
        sickles = self.sickles + other.sickles
        knuts = self.knuts + other.knuts
        return Vault(galleons, sickles, knuts)


potter = Vault(100, 50, 25)
print(potter)

weasley = Vault(25, 50, 100)
print(weasley)

total = potter + weasley
print(total)
```

Observe como o método __str__ retorna uma string formatada. Além disso, observe como o método __add__ permite a adição dos valores de dois vaults. self é o que está à esquerda do operando +. outro é o que está certo do +.

RESUMINDO

Agora você aprendeu um novo nível de capacidade por meio da programação orientada a objetos.

- Programação orientada a objetos
- Aulas
- elevação
- Métodos de classe
- Métodos estáticos
- Herança
- Sobrecarga do Operador